

Question 1:

(i)

```
// An education institute has to maintain a list of courses it
// offers.
// There is a need to access the list of courses frequently. The
// existing courses are rarely removed, while new courses are
// often added. Considering this information, use an appropriate
// data structure to implement a system that allows for adding new
// course, removing existing course from the system, and listing
// courses.
#include <stdio.h>
#include <string.h>

struct Course {
    char id[10];
    char name[50];
    int semester;
};

void addCourse(struct Course courses[], int *count){
    printf("Enter course ID: ");
    scanf("%s", courses[*count].id);
    printf("Enter course name: ");
    scanf("%s", courses[*count].name);
    printf("Enter course semester: ");
    scanf("%d", &courses[*count].semester);
    (*count)++;
}

void removeCourse(struct Course courses[], int *count, const char
*id){
    for(int i = 0; i < *count; i++){
        if(strcmp(courses[i].id, id) == 0){
            for(int j = i; j < *count - 1; j++){
                courses[j] = courses[j + 1];
            }
            (*count)--;
            printf("Course with ID %s removed.\n", id);
            return;
        }
    }
    printf("Course with ID %s not found.\n", id);
}

void listCourses(struct Course courses[], int count){
    printf("List of Courses:\n");
    for(int i = 0; i < count; i++){
```

```

        printf("ID: %s, Name: %s, Semester: %d\n", courses[i].id,
courses[i].name, courses[i].semester);
    }
}

int main(){
    struct Course courses[100];
    int count = 0;
    int choice;

    while(1){
        printf("\n1. Add Course\n2. Remove Course\n3. List
Courses\n4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch(choice){
            case 1:
                addCourse(courses, &count);
                break;
            case 2: {
                char id[10];
                printf("Enter course ID to remove: ");
                scanf("%s", id);
                removeCourse(courses, &count, id);
                break;
            }
            case 3:
                listCourses(courses, count);
                break;
            case 4:
                return 0;
            default:
                printf("Invalid choice. Please try again.\n");
        }
    }

    return 0;
}

```

(ii) Leetcode 1. Two Sum

```
class Solution {
    public int[] twoSum(int[] nums, int target) {
        for(int i = 0; i < nums.length; i++){
            for(int j = i + 1; j < nums.length; j++){
                if((nums[i] + nums[j]) == target)
                    return new int[]{i,j};
            }
        }
        return null;
    }
}
```

(ii) Leetcode 74. Search a 2D matrix

```
class Solution {
    public boolean searchMatrix(int[][] matrix, int target) {
        int m = matrix.length;
        int n = matrix[0].length;
        int l = 0;
        int r = m * n - 1;

        while(l <= r){
            int mid = (l + r) / 2;
            int i = mid / n;
            int j = mid % n;

            if(matrix[i][j] == target)
                return true;
            if(matrix[i][j] < target)
                l = mid + 1;
            else
                r = mid - 1;
        }
        return false;
    }
}
```